

Documentação do Projeto: NavGraph

Sistema de Navegação Primitivo baseado no Algoritmo de Dijkstra

INF/UFG 2026-1-AED2

Prof. André L. Moura

5 de maio de 2026

Integrantes

- (Nome completo em ordem alfabética)
- (Nome completo em ordem alfabética)
- (Nome completo em ordem alfabética)
- (Nome completo em ordem alfabética)

Conteúdo

1	Introdução	3
1.1	Objetivo do Documento	3
1.2	Escopo do Projeto e Objetivos	3
1.3	Visão Geral do Sistema	3
2	Arquitetura do Sistema	3
2.1	Visão Arquitetural	3
2.2	Descrição das Camadas	3
2.2.1	Camada de Apresentação (UI)	3
2.2.2	Camada de Domínio (Core)	3
2.2.3	Módulos Utilitários	3
2.3	Tecnologias Utilizadas	4
3	Detalhamento dos Módulos	4
3.1	Módulo Core ('src/core')	4
3.2	Módulo UI ('src/ui')	4
3.3	Módulos Utilitários ('src/utills')	4
4	Funcionalidades Detalhadas	4
4.1	Requisitos Funcionais (RF)	4
4.2	Requisitos Não Funcionais (RNF)	4
5	Formatos de Dados de Entrada	5
5.1	Arquivo '.txt' (Lista de Adjacência)	5
5.2	Arquivo '.osm' (OpenStreetMap)	5
5.3	Arquivo '.poly'	5
6	Guia de Instalação e Execução	5
6.1	Pré-requisitos	5
6.2	Passos para Instalação	5
6.3	Comandos para Execução e Testes	5
7	Guia de Uso da Aplicação	5
7.1	Interação com o Canvas	5
7.2	Encontrar Caminho Mínimo	5
7.3	Importação, Edição e Exportação	5

1 Introdução

1.1 Objetivo do Documento

Este documento apresenta a especificação técnica, arquitetural e funcional do sistema NavGraph, desenvolvido como projeto final da disciplina de Algoritmos e Estruturas de Dados 2 (AED2)[cite: 3]. O sistema visa encontrar o caminho mais curto entre dois pontos geográficos utilizando grafos e o algoritmo de Dijkstra[cite: 1].

1.2 Escopo do Projeto e Objetivos

O NavGraph é um sistema com interface visual que permite importar mapas reais, convertê-los em grafos e calcular rotas indicativas[cite: 1]. Os objetivos principais incluem o uso eficiente de estruturas de dados (Lista de Adjacência e Heap Mínima) para garantir alto desempenho em grafos com milhares de vértices[cite: 1].

1.3 Visão Geral do Sistema

O software é composto por um motor de cálculo em Java (Módulo Core) e uma interface interativa em Python (Módulo UI). O sistema processa arquivos de mapas (.osm, .poly, .txt) para visualização e manipulação de redes viárias[cite: 2, 3].

2 Arquitetura do Sistema

2.1 Visão Arquitetural

A arquitetura é modular e baseada em camadas, separando a lógica de negócio (algoritmos) da camada de interação[cite: 3]. A comunicação entre Python e Java ocorre via execução de subprocessos com troca de dados estruturados.

2.2 Descrição das Camadas

2.2.1 Camada de Apresentação (UI)

Desenvolvida em Python, é responsável pela renderização do grafo no canvas, gestão de eventos de mouse e exibição de estatísticas[cite: 1, 3].

2.2.2 Camada de Domínio (Core)

Implementada em Java, contém a lógica do algoritmo de Dijkstra e a gestão da Fila de Prioridade (Heap Mínima)[cite: 1].

2.2.3 Módulos Utilitários

Incluem parsers de arquivos e rotinas de exportação de imagem[cite: 3].

2.3 Tecnologias Utilizadas

- **Linguagens:** Java e Python[cite: 1].
- **Algoritmo:** Dijkstra com Fila de Prioridade (Heap Mínima)[cite: 1].
- **Dados:** OpenStreetMap (OSM) e formato POLY[cite: 2].

3 Detalhamento dos Módulos

3.1 Módulo Core ('src/core')

O motor de cálculo em Java utiliza uma **Lista de Adjacência** para armazenamento eficiente ($O(V+E)$) e uma **Heap Mínima** para extração do menor custo em $O(\lg V)$ [cite: 1].

3.2 Módulo UI ('src/ui')

Interface construída para ser intuitiva (RNF06) [cite: 1], permitindo selecionar origem/destino com cliques[cite: 1].

3.3 Módulos Utilitários ('src/utlis')

Contém funções para conversão de coordenadas UTM e captura de tela (RF08)[cite: 1, 2].

4 Funcionalidades Detalhadas

4.1 Requisitos Funcionais (RF)

Conforme especificado no edital[cite: 1]:

- **RF01:** Importação de mapas reais e conversão para grafos.
- **RF02:** Enumeração de vértices e rotulação de pesos (distâncias).
- **RF03:** Seleção de origem/destino com cores distintas.
- **RF04:** Cálculo e exibição da rota do menor caminho.
- **RF05:** Criação e edição de grafos via mouse.
- **RF06:** Suporte a vias direcionadas/não direcionadas (mão única/dupla).
- **RF07:** Exibição de estatísticas de execução (tempo, nós, custo).
- **RF08:** Cópia da imagem do grafo para área de transferência.

4.2 Requisitos Não Funcionais (RNF)

- **RNF03/05:** Otimização para grandes grafos e uso eficiente de memória[cite: 1].
- **RNF04:** Tempo de resposta inferior a 2 segundos para ~ 500 nós[cite: 1].

5 Formatos de Dados de Entrada

5.1 Arquivo '.txt' (Lista de Adjacência)

Entrada simplificada para grafos manuais[cite: 3].

5.2 Arquivo '.osm' (OpenStreetMap)

Dados geográficos brutos exportados do OpenStreetMap[cite: 2, 3].

5.3 Arquivo '.poly'

Formato gerado pelo conversor que utiliza coordenadas UTM e escala reduzida[cite: 2].

6 Guia de Instalação e Execução

6.1 Pré-requisitos

Java JRE/JDK 11+ e Python 3.8+ instalados no sistema[cite: 1].

6.2 Passos para Instalação

Descompactar o arquivo `PF_NomeAluno.zip` e localizar o instalador[cite: 1].

6.3 Comandos para Execução e Testes

Utilizar o arquivo executável ou comando `java -jar` para o core e `python app.py` para a UI.

7 Guia de Uso da Aplicação

7.1 Interação com o Canvas

Utilize o clique esquerdo para selecionar vértices e o modo de edição para manipular arestas[cite: 1].

7.2 Encontrar Caminho Mínimo

Selecione origem e destino e clique em “Traçar Menor Caminho”[cite: 1].

7.3 Importação, Edição e Exportação

Importe arquivos `.poly/osm` via menu de arquivos e use a função de cópia (Ctrl+C) para imagem[cite: 1].